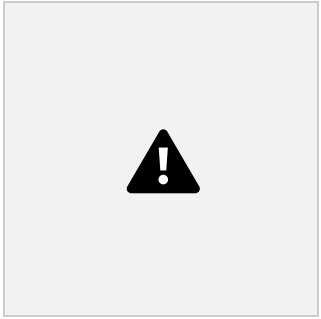
Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Sistemas de Bases de Datos 2



Manual Técnico

No. Registro - Nombre
201831260 - Michael Kristopher Marín Reyes

Quetzaltenango, Octubre de 2026.

Índice

1. Objetivo y decisiones arquitectónicas....5
2. Stack y versiones....7
3. Organización del repositorio....8
4. Instalación del entorno....9
4.1 Herramientas base: GNU/Linux Debian/Ubuntu....9
4.2 Node.js, React y Vite....9
4.3 Docker y Compose....9
4.4 Instalar Spring Boot: qué significa en este proyecto....10
5. Ejecutar el proyecto existente....10
5.1 Obtener el código y preparar configuración....10
5.2 Iniciar bases desde la raíz....10
5.3 Backend en una terminal....10
5.4 Frontend en otra terminal....11
5.5 Empaquetar....11
6. Crear el proyecto desde cero....11
6.1 Generar el backend con Spring Initializr....12
6.2 Crear el frontend con React y Vite....12
7. Configuración y seguridad....13
8. Contrato HTTP....14
8.1 Consulta ejecutada y ejecución editada....16
9. ETL y reglas de negocio....17
10. Modelo de persistencia....18
11. Inventario del backend: clases y métodos....19
ProyectoFinalDb2Application....20
AsyncConfig....20
DotenvEnvironmentPostProcessor....20
MongoIndexInitializer....20
Neo4jConfig....21
Neo4jSchemaInitializer....21
WebConfig....21
BusquedaController....22
CargaController....22
CatalogoController....23
ConsultaMongoController....23
GrafoController....23
HealthController....24
SavedQueryController....24
ApiError....25
CargaReporteDto....25

CustomQueryRequest 25
GrafoDto 26
QueryResult 26
QueryView 26
SavedQueryRequest 26
ApiException 26
GlobalExceptionHandler 27
CypherCatalog 28
EvaluationRepository 28
CargaJob 29
CargaService 31
CatalogService 32
ConsultaEditorService 32
CustomQueryPolicy 33
EtlService 33
GraphService 34
MongoQueryService 35
SavedQueryService 37
SearchService 38
TransformationService 39
CsvSupport 40
MongoJson 41
12. Inventario del frontend 41
src/App.jsx 41
src/api/client.js 42
src/components/Cargando.jsx 42
src/components/ConsultaEjecutada.jsx 42
src/components/ConsultasGuardadas.jsx 42
src/components/EstadoError.jsx 42
src/components/FichaAspirante.jsx 42
src/components/FiltroBar.jsx 43
src/components/Grafica.jsx 43
src/components/GrafoCanvas.jsx 43
src/components/KpiCard.jsx 43
src/components/Layout.jsx 44
src/components/Modal.jsx 44
src/components/TablaResultados.jsx 44
src/components/UploadDragDrop.jsx 44
src/hooks/useCarga.js 44
src/hooks/useCatalogos.js 44
src/hooks/useConsultas.js 45

src/main.jsx..... 45
src/pages/Buscador.jsx.....45
src/pages/Carga.jsx.....45
src/pages/Consultas.jsx.....45
src/pages/Dashboard.jsx.....46
src/pages/Error404.jsx.....46
src/pages/ExploradorGrafos.jsx.....46
src/utils/csvValidation.js.....46
src/utils/formato.js.....46
src/workers/csvValidation.worker.js.....47
13. Pruebas y verificación.....47
14. Despliegue y operación..... 48
15. Limitaciones y evolución..... 49
16. Referencias y trazabilidad.....50
17. Consultas definidas..... 50
18. Repositorio.....50

Manual técnico

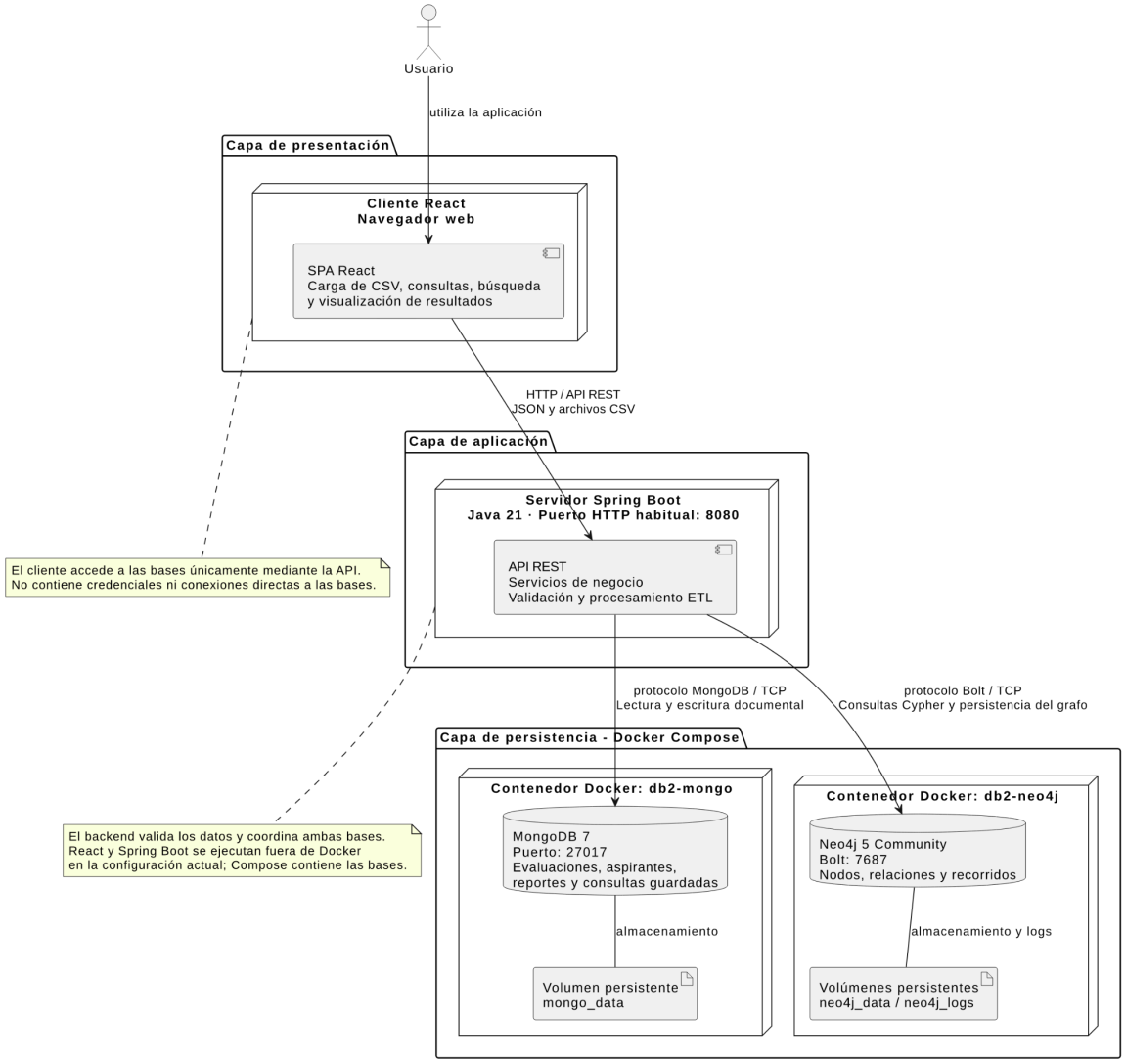
La documentación que se presenta indica la estructura y cómo funciona la aplicación para el análisis de datos de: Pruebas específicas de aspirantes a Ingeniería, Universidad de San Carlos de Guatemala. Alcance: backend Java/Spring Boot, frontend React, persistencia MongoDB y Neo4j, ETL, API, configuración, instalación, inventario de clases y métodos, pruebas y operación.

1. Objetivo y decisiones arquitectónicas

El sistema carga archivos CSV de pruebas específicas, valida y normaliza sus registros, conserva evaluaciones documentales y un modelo de grafo, y permite consultas, búsqueda y visualización. La API es la única frontera de acceso del navegador a las bases de datos. La arquitectura es una SPA desacoplada de un monolito modular Spring Boot con persistencia.

Capa	Responsabilidad	Regla de dependencia
Presentación React	Navegación, formularios, revisión local de CSV, tablas, gráficas y grafo.	Consume HTTP; nunca accede directamente a drivers de bases.
API REST	Adaptar peticiones y respuestas, validar parámetros y gestionar códigos HTTP.	Delega reglas a servicios.
Servicios	ETL, consultas analíticas, búsquedas, biblioteca administrativa.	Orquesta repositorio MongoTemplate y driver Neo4j.
Persistencia	Documentos, índices, agregaciones y operaciones Cypher.	MongoDB y Neo4j tienen funciones diferentes; no hay transacción distribuida.
Configuración transversal	.env, CORS, executor, drivers, índices y excepciones.	No almacenar secretos en YAML ni JavaScript distribuido.

Adjunto a este documento se encuentran las imagenes en formato png y svg para una mejor visualización



Conexión	Protocolo / dirección	Propósito
Navegador → frontend	HTTP en desarrollo; HTTPS recomendado en producción	Descarga de HTML, JavaScript y CSS de la SPA.
SPA → API	Axios; /api por proxy o URL pública configurada	JSON para consultas y reportes; multipart para archivos. No usa WebSocket.
Backend → MongoDB	Protocolo MongoDB sobre TCP 27017 en el Compose local	Consultas documentales, índices, auditoría y biblioteca de consultas.
Backend → Neo4j	Bolt sobre TCP 7687 en el Compose local	Sesiones y transacciones locales con el driver oficial.
Administrador → Neo4j Browser	HTTP 7474 local	Herramienta administrativa separada; no es el camino de acceso del frontend.

Contenedores → volúmenes	mongo_data, neo4j_data y neo4j_logs	Persistencia fuera del ciclo de vida del contenedor.

Flujo de lectura: página React → hook/Axios → proxy HTTP → controlador → servicio → MongoTemplate o Driver Neo4j → JSON normalizado → tabla/gráfica/grafo. Las páginas no reciben conexiones ni secretos de las bases.
Flujo de carga: selección → Worker de revisión → resumen local → confirmación → multipart → 202 con ID → ETL asíncrono → polling HTTP cada 1500 ms → reporte final. La revisión local mejora la experiencia, pero no determina la validez de lo que Java almacena.

2. Stack y versiones

Elemento	Versión declarada / procedencia	Uso
Java	21, propiedad java.version en pom.xml	Lenguaje y nivel de compilación; se recomienda JDK 21.
Spring Boot	4.1.2, spring-boot-starter-parent	Autoconfiguración, Spring MVC y aplicación ejecutable.
Maven	3.10.0; Maven Wrapper 3.3.4	Compilación y gestión de dependencias; wrapper incluido.
OpenCSV	5.9	Extracción CSV por registros.
dotenv-java	3.2.0	Lectura de configuración privada antes de crear drivers.
Spring Data MongoDB / driver Neo4j	Gestionados por el BOM de Spring Boot	Acceso documental y transacciones locales de grafo.
MongoDB	Imagen mongo:7	Documentos, agregaciones, índices y auditoría.
Neo4j	Imagen neo4j:5-community	Modelo de relaciones, Cypher y recorridos.
Node.js / npm	Recomendados Node 22 LTS y npm 10 o superior	Herramientas de desarrollo; no se fijan con engines en package.json.
Docker / Compose	Docker Engine compatible y plugin Compose v2	Solo ejecuta bases en el compose actual.

Paquete frontend	Declaración	Versión resuelta en lockfile

axios	1.7.9	1.7.9
papaparse	^5.5.3	5.5.3
react	18.3.1	18.3.1
react-dom	18.3.1	18.3.1
react-force-graph-2d	1.27.0	1.27.0
react-router-dom	6.30.1	6.30.1
recharts	2.15.4	2.15.4
@vitejs/plugin-react	4.3.4	4.3.4
vite	5.4.14	5.4.14

Las versiones anteriores son las del proyecto, no una afirmación de que sean las últimas disponibles. npm ci respeta package-lock.json. Las etiquetas Docker por versión mayor pueden recibir actualizaciones: para producción deben fijarse versiones o digests tras validación. Las dependencias transitivas Java se inspeccionan con ./mvnw dependency:tree; no se inventan versiones que administra Spring Boot.

3. Organización del repositorio

```
ProyectoFinal/
  docker-compose.yml              Bases de datos y volúmenes
  proyecto-final-db2/             Backend Java y Maven Wrapper
    pom.xml
    .mvn/wrapper/maven-wrapper.properties
    src/main/java/com/db2/proyectofinaldb2/
      config/ controller/ dto/ exception/
      neo4j/ repository/ service/ util/
    src/main/resources/application.yml
    src/main/resources/META-INF/spring.factories
    src/test/java/                Pruebas locales
  frontend-proyecto-final-db2/
    package.json / package-lock.json / vite.config.js
    src/api/ src/hooks/ src/pages/ src/components/
    src/utils/ src/workers/
  infra/mongo/.env                Configuración privada
  infra/neo4j/.env                Configuración privada
  etl/muestras/                   CSV de referencia locales
  etl/validar_dataset.md          Guía de validación
  documentacion/                  Diagramas WSD y este manual DOCX
  docs/                           Markdown y notas locales ignorados por Git
```

4. Instalación del entorno

4.1 Herramientas base: GNU/Linux Debian/Ubuntu

Los siguientes comandos requieren privilegios administrativos y acceso a red. Verificar que la distribución ofrezca OpenJDK 21 antes de ejecutarlos; en otra distribución instalar el JDK equivalente desde su proveedor oficial.

```
sudo apt-get update
sudo apt-get install -y openjdk-21-jdk git curl unzip
java -version
javac -version
git --version
```

Configurar JAVA_HOME hacia el JDK instalado y añadir su directorio bin al PATH si el sistema no lo hace. El IDE y Maven deben utilizar el mismo JDK. No es necesario instalar Maven globalmente si se usa el wrapper.

4.2 Node.js, React y Vite

Instalar Node.js 22 LTS desde el proveedor oficial o mediante un gestor de versiones ya instalado. React no es un programa global: es una dependencia npm del frontend. No se usa npm install -g react ni Create React App para este proyecto.

```
# Si nvm ya está instalado en el equipo:
nvm install 22
nvm use 22
node --version
npm --version
```

Si nvm no está disponible, instalar primero Node desde https://nodejs.org/ siguiendo las instrucciones del sistema operativo. Evitar ejecutar scripts remotos no inspeccionados con permisos de administrador.

4.3 Docker y Compose

Instalar Docker Engine y el plugin Docker Compose desde la guía oficial correspondiente al sistema: https://docs.docker.com/engine/install/. En sistemas con repositorio oficial Docker ya configurado:

```
sudo apt-get install -y docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
docker --version
docker compose version
```

La pertenencia al grupo docker equivale a privilegios elevados; usar sudo cuando corresponda. Docker Desktop es una alternativa en Windows/macOS. Este manual usa sintaxis de shell Linux; en Windows el wrapper es mvnw.cmd y las rutas deben adaptarse.

4.4 Instalar Spring Boot: qué significa en este proyecto

Spring Boot se instala como dependencias Maven declaradas en pom.xml; no requiere un servidor de aplicaciones externo ni una instalación global del CLI. ./mvnw descarga la distribución Maven fijada y resuelve Spring Boot 4.1.2 desde los repositorios configurados. Se inicia con spring-boot:run o como JAR ejecutable. Un IDE con soporte Spring es opcional, no una dependencia de ejecución.

5. Ejecutar el proyecto existente

5.1 Obtener el código y preparar configuración

```
git clone <URL_REAL_DEL_REPOSITORIO>
cd ProyectoFinal
mkdir -p infra/mongo infra/neo4j
touch proyecto-final-db2/.env frontend-proyecto-final-db2/.env
touch infra/mongo/.env infra/neo4j/.env
chmod 600 proyecto-final-db2/.env frontend-proyecto-final-db2/.env
infra/mongo/.env infra/neo4j/.env
```

Los comandos touch solo crean archivos: NO configuran valores. Completar cada .env con un editor y valores propios según la sección 7. No iniciar las aplicaciones con archivos vacíos. Si ya existe configuración, preservarla; nunca copiar secretos al DOCX ni al repositorio.

5.2 Iniciar bases desde la raíz

```
docker compose config --quiet
docker compose up -d
docker compose ps
docker logs db2-mongo --tail 30
docker logs db2-neo4j --tail 30
```

Esperar a que las bases estén listas. Compose publica 27017, 7474 y 7687 y conserva mongo_data, neo4j_data y neo4j_logs. No ejecutar docker compose down -v: elimina volúmenes. Para detener sin borrar datos usar docker compose down.

5.3 Backend en una terminal

```
cd proyecto-final-db2
```

```
chmod +x mvnw
./mvnw -v
./mvnw spring-boot:run
```

Ejecutar una sola instancia del backend por puerto. Si el IDE ya lo inició, no lanzar otro proceso. El puerto se define con SERVER_PORT en su .env; 8080 es la referencia local habitual. Los inicializadores intentan crear índices y restricciones; la salud debe comprobarse después del arranque.

curl --fail-with-body http://localhost:8080/api/salud

5.4 Frontend en otra terminal

```
cd frontend-proyecto-final-db2
npm ci
npm run dev -- --host 127.0.0.1
```

Abrir la URL que imprime Vite, habitualmente http://127.0.0.1:5173. La configuración exige API_PROXY_TARGET y FRONTEND_PORT. Las peticiones /api pasan por el proxy durante desarrollo. localhost y 127.0.0.1 son orígenes distintos: configurar CORS y URLs coherentemente.

5.5 Empaquetar

```
# Dentro de proyecto-final-db2/
./mvnw test package
java -jar target/proyecto-final-db2-0.0.1-SNAPSHOT.jar
# Dentro de frontend-proyecto-final-db2/
npm run build
npm run preview
```

No ejecutar el JAR si spring-boot:run continúa usando el mismo puerto. npm run preview sirve para inspección local del bundle; no es el servidor de producción. El .env del backend se resuelve por el directorio de trabajo o por el entorno; no va incluido en el JAR.

6. Crear el proyecto desde cero

Esta sección explica reconstrucción en un directorio VACÍO y separado. Para usar el proyecto existente no repetir scaffolding sobre sus carpetas. El código de negocio, las políticas ETL y los diagramas no los genera Spring Initializr ni Vite.

6.1 Generar el backend con Spring Initializr

En https://start.spring.io/ seleccionar Maven, Java, Spring Boot 4.1.2, Java 21, group com.db2, artifact proyecto-final-db2, package com.db2.proyectofinaldb2 y empaquetado JAR. Elegir Spring Web/MVC, Spring Data MongoDB, Neo4j, Validation y Actuator; DevTools y Lombok son opcionales de desarrollo.

```
# En un directorio de reconstrucción independiente:
curl --fail --get https://start.spring.io/starter.zip \
  --data-urlencode type=maven-project \
  --data-urlencode language=java \
  --data-urlencode bootVersion=4.1.2 \
  --data-urlencode javaVersion=21 \
  --data-urlencode groupId=com.db2 \
  --data-urlencode artifactId=proyecto-final-db2 \
  --data-urlencode packageName=com.db2.proyectofinaldb2 \
  --data-urlencode packaging=jar \
  --data-urlencode
dependencies=web,data-mongodb,data-neo4j,validation,actuator \
  --output backend-inicial.zip
unzip backend-inicial.zip -d backend-reconstruido
```

Initializr cambia su catálogo con el tiempo. Si no ofrece la versión o un identificador de dependencia, consultar https://start.spring.io/metadata/client y usar la interfaz; no reemplazar silenciosamente la versión. El pom.xml del repositorio es la referencia de reproducción: sus starters concretos son webmvc, data-mongodb, neo4j, validation y actuator, con los starters de test correspondientes. Añadir OpenCSV 5.9 y dotenv-java 3.2.0, el postprocesador dotenv y su registro en META-INF/spring.factories. Mantener Java 21, Maven Wrapper y los paquetes descritos en este manual.

```
# En backend-reconstruido/, una vez ajustado el pom:
./mvnw dependency:resolve
./mvnw test package
./mvnw spring-boot:run
```

6.2 Crear el frontend con React y Vite

```
# En el directorio de reconstrucción, NO sobre el frontend actual:
npm create vite@5 frontend-reconstruido -- --template react
cd frontend-reconstruido
npm install --save-exact react@18.3.1 react-dom@18.3.1 react-router-dom@6.30.1
axios@1.7.9 recharts@2.15.4 react-force-graph-2d@1.27.0 papaparse@5.5.3
npm install --save-dev --save-exact vite@5.4.14 @vitejs/plugin-react@4.3.4
```

El scaffold puede traer dependencias adicionales como ESLint. Ajustar package.json y scripts para que coincidan con el repositorio, crear src/pages, src/components, src/hooks, src/api, src/utils y src/workers; montar App dentro de BrowserRouter, configurar el proxy y preparar el .env. El scaffold por sí solo no contiene las pantallas ni el ETL. El proyecto actual declara PapaParse con rango ^5.5.3; package-lock.json determina la versión exacta instalada.

```
npm pkg set scripts.dev="vite" scripts.build="vite build" scripts.preview="vite preview"
npm pkg set 'scripts.test=node --test src/utils/*.test.js'
npm run dev
npm run build
```

7. Configuración y seguridad

Se usan cuatro archivos .env privados. Crear con sintaxis NOMBRE=valor, sin export. Las variables del sistema y CLI tienen prioridad sobre dotenv. Los ejemplos no incluyen contraseñas por defecto: cada instalación debe definir sus propios valores.

Archivo	Claves / propósito
proyecto-final-db2/.env	SERVER_PORT; SERVER_ERROR_INCLUDE_MESSAGE; SPRING_MONGODB_URI; SPRING_NEO4J_URI; SPRING_NEO4J_AUTHENTICATION_USERNAME; SPRING_NEO4J_AUTHENTICATION_PASSWORD.
proyecto-final-db2/.env	SPRING_SERVLET_MULTIPART_MAX_FILE_SIZE; SPRING_SERVLET_MULTIPART_MAX_REQUEST_SIZE; APP_ETL_BATCH_SIZE (1..10000); APP_ETL_TEMP_DIR; APP_CORS_ALLOWED_ORIGINS; LOGGING_LEVEL_COM_DB2_PROYECTOFINALDB2.
proyecto-final-db2/.env	APP_QUERIES_ADMIN_KEY (mínimo 32 caracteres); APP_QUERIES_MAX_ROWS; APP_QUERIES_TIMEOUT_SECONDS.
frontend-proyecto-final-db2/.env	VITE_API_URL (pública, normalmente ruta /api); API_PROXY_TARGET (destino de desarrollo); FRONTEND_PORT.
infra/mongo/.env	MONGO_INITDB_DATABASE: nombre elegido para inicialización. La base usada por

	el backend debe declararse explícitamente en SPRING_MONGODB_URI.
infra/neo4j/.env	NEO4J_AUTH y opciones de memoria requeridas por la imagen. Deben ser coherentes con usuario y contraseña del backend.

- DotenvEnvironmentPostProcessor busca proyecto-final-db2/.env al ejecutar desde la raíz o .env en el directorio actual. Se registra en META-INF/spring.factories para actuar antes de crear beans.
- VITE_* se incrusta en JavaScript público. Nunca poner ahí URIs con credenciales, contraseñas de bases ni la clave administrativa.
- La clave administrativa se introduce por el usuario y se conserva en memoria React; viaja por X-Query-Admin-Key. No es autenticación general de la aplicación.
- CORS no es autenticación. Los endpoints de carga y lectura no incorporan un sistema general de usuarios/roles. Añadir autorización, HTTPS y límites en el gateway antes de exponerlos públicamente.
- MongoDB del Compose actual no activa autenticación mediante MONGO_INITDB_DATABASE. No confundir inicializar el nombre con crear usuarios ni proteger la base. Restringir puertos/red y diseñar usuarios mínimos para producción.
- Cambiar NEO4J_AUTH no rota una contraseña persistida en un volumen existente. Rotar por el mecanismo de Neo4j y actualizar la configuración coherentemente; no borrar volúmenes para cambiarla.
- El archivo .env no revoca secretos publicados antes. Rotar credenciales expuestas; la limpieza del historial es una operación independiente.
- El editor de consultas ejecutadas aplica solo lectura, un límite de 200 filas y 10 segundos; no sustituye autenticación ni autorización y debe restringirse antes de exponer la API.
- No guardar datos originales o rechazados reales en Git. Sanitizar logs y reportes antes de compartirlos.

8. Contrato HTTP

Método y ruta (prefijo /api)	Entrada	Salida / efecto
GET /salud	Sin parámetros.	Estado independiente de MongoDB/Neo4j y conteos. Puede responder salud degradada.
POST /cargas/archivos	multipart/form-data; files, uno o varios CSV.	202 Accepted con CargaReporteDto; inicia procesamiento local asíncrono.

GET /cargas	Sin parámetros.	Hasta 50 reportes persistidos.
GET /cargas/{id}	ID de carga.	Reporte en memoria si existe; en otro caso persistido.
GET /cargas/{id}/rechazados	page>=0; size acotado 1..500.	Rechazos auditados y paginados.
GET /consultas/mongo/{numero}	1..20; anio, materia, carrera, depto, tipo, limit y correlativo según consulta.	Filas JSON. C11/C15 requieren correlativo.
POST /consultas/mongo/6/resumen	Sin cuerpo.	Reconstruye resumen_carrera: este endpoint modifica datos derivados.
GET /grafo/consultas/{numero}	1..7; filtros por año/carrera/departamento/materia/tipo y limit.	Filas Cypher; G7 requiere departamento y carrera.
GET /grafo/visualizacion	Filtros y limit.	GrafoDto con nodes y links para ForceGraph2D.
GET /grafo/nodo/{id}	elementId codificado en URL.	Propiedades y relaciones del nodo.
GET /buscar	tipo y q.	Resultados por aspirante, carrera, materia, institución, municipio o departamento.
GET /aspirantes/{correlativo}	Identificador textual codificado.	Ficha embebida completa.
GET /aspirantes/{correlativo}/historial	Correlativo textual.	Evaluaciones ordenadas.
GET /catalogos/{tipo}	materias, carreras, departamentos, municipios, tipos o anios.	Opciones de filtros con conteos.
GET /consultas/guardadas	motor y X-Query-Admin-Key.	Definiciones persistidas del motor.
POST /consultas/guardadas	JSON nombre, descripcion, motor, codigo y clave en cabecera.	201 Created; valida y guarda definición.
POST /consultas/guardadas/{id}/ejecutar	Clave y confirmarEscritura=true si corresponde.	Resultado y métricas de ejecución; puede modificar una base.

Respuestas de error: ApiError centraliza timestamp, status, error, mensaje y ruta. Según el caso se usan 400 (entrada inválida), 401 (clave incorrecta), 404 (recurso no encontrado), 408 (timeout MongoDB personalizado), 409 (falta confirmación de escritura), 413 (multipart demasiado

grande), 500 (error inesperado) y 503 (servicio no disponible). La UI normaliza mensaje mediante interceptor Axios. Un 202 solo confirma aceptación del trabajo, no éxito de persistencia: consultar hasta estado final.

```
# Consulta de salud, sin cargar datos:
curl --fail-with-body http://localhost:8080/api/salud
# EJEMPLO que modifica bases: usar solo con datos aprobados para carga.
curl --fail-with-body -X POST -F 'files=@/ruta/dataset.csv'
http://localhost:8080/api/cargas/archivos
# Sustituir el identificador devuelto:
curl --fail-with-body "http://localhost:8080/api/cargas/<ID_CARGA>"
```

La biblioteca MongoDB admite JSON estructurado (find, aggregate, insertMany, updateMany y deleteMany), no JavaScript mongosh; limita colecciones a evaluaciones, aspirantes y resumen_carrera y bloquea operadores de ejecución de JavaScript. Neo4j admite una sentencia de datos, sin administración ni procedimientos; EXPLAIN valida y clasifica antes de guardar/ejecutar. Las escrituras requieren además confirmación explícita. No constituyen un sandbox general para usuarios no confiables.

8.1 Consulta ejecutada y ejecución editada

Las consultas analíticas GET /api/consultas/mongo/{1..20} y GET /api/grafo/consultas/{1..7} responden con { filas, consulta }. El arreglo filas alimenta la tabla o gráfica y consulta es un objeto QueryView con motor, codigo, nota opcional y parametros, este último solo en Cypher. El cliente muestra esa consulta al final del resultado.

El visor permite editar codigo y volver a ejecutarlo contra POST /api/consultas/mongo/ejecutar (cuerpo { codigo }) o POST /api/grafo/ejecutar (cuerpo { codigo, parametros }). Ambos endpoints aceptan solo lectura, aplican un máximo de 200 filas y un timeout de 10 segundos por defecto, y responden { filas, truncado, limiteFilas, duracionMs }. El resultado editado se muestra en una tabla aparte sin reemplazar la visualización original.

En MongoDB, codigo debe ser un objeto JSON con operacion find o aggregate; se rechazan $out, $merge, insertMany, updateMany, deleteMany y las colecciones fuera de evaluaciones, aspirantes y resumen_carrera. En Cypher se admite una sola sentencia de datos sin CALL ni administración; parametros conserva los valores originales, incluidos los nulos.

```
curl -X POST -H 'Content-Type: application/json' \
  -d '{"codigo":"{\"operacion\":\"find\",\"coleccion\":\"aspirantes\",\"filtro\":{}}"}' \
  http://localhost:8080/api/consultas/mongo/ejecutar
curl -X POST -H 'Content-Type: application/json' \
  -d '{"codigo":"MATCH (m:Materia) RETURN m.nombre AS materia LIMIT
$limite","parametros":{"limite":5}}' \
  http://localhost:8080/api/grafo/ejecutar
```

9. ETL y reglas de negocio

- Recepción: CargaController delega a CargaService. Se valida extensión y archivo no vacío, se copia a una carpeta temporal por carga con nombre saneado y se prepara CargaJob.
- Programación: se verifican índices MongoDB, se persiste el inicio y un executor de dos hilos ejecuta EtlService. El hilo HTTP no procesa todo el CSV.
- Extracción: CsvSupport detecta coma, punto y coma o tabulador; OpenCSV lee UTF-8 estrictamente, admite comillas y limita registros multilínea a 100 líneas. Se valida la estructura y se omiten filas totalmente vacías.
- Transformación: encabezados con alias y tildes; rechaza duplicidad semántica. Cada registro debe tener la misma cantidad de columnas que el encabezado. Valida campos de hasta 4096 caracteres y rechaza controles no permitidos.
- Carga: inserta lotes unordered en MongoDB; distingue insertados, duplicados y fallos conocidos por índice. Solo filas aceptadas o duplicadas pasan a Neo4j, permitiendo reintentar el grafo.
- Derivados y cierre: reconstruye aspirantes cuando hubo filas confirmadas, incluso si después falló Neo4j. Persiste reporte y limpia temporales aunque falle el guardado final. No reconstruye automáticamente resumen_carrera en cada carga.

Campo	**Regla autoritativa**
correlativo_aspirante	Texto obligatorio. Conserva ceros iniciales. No es ID único de evaluación; una persona puede tener muchas materias, años e intentos.
materia	Obligatoria; limpieza de espacios y mayúsculas.
anio_ingreso	Entero entre 1990 y 2100.
oportunidad	Entero positivo; se mapea desde numero_de_fecha_de_evaluacion y alias.
aprobacion	Catálogo cerrado: APROBADO/SI/SÍ/1 y NO APROBADO/NO/0. Normaliza tildes, espacios, _ y -. Desconocidos se rechazan; no usa subcadenas.
anio_nacimiento	Vacío, SIN REGISTRO, formato inválido o fuera de 1900..año actual: null.
edad_ingreso	Se recalcula por diferencia de años; solo se conserva entre 10 y 80.
fecha_asignacion	Fecha válida en formatos admitidos, UTC a medianoche. Vacía o imposible: null; no se ajusta 31/02 a otro día.
carrera_objetivo	Mayúsculas; vacío o null de origen se convierte a NINGUNO. NINGUNO se conserva como categoría explícita.

sexo	M/F reconocidos por alias; desconocidos: DESCONOCIDO.
departamento_ie / municipio_ie / tipo_ie	Campos geográficos en mayúsculas, vacíos SIN REGISTRO. Tipo institucional reconoce PUBLICO/PRIVADO y equivalentes.
hash_fila / aprobado / edad	Siempre recalculados en Java, incluso si el cliente aporta valores.
archivo_origen / fila_origen / archivo_carga	Conserva procedencia auxiliar válida y registra archivo realmente subido. La procedencia proporcionada por el cliente no es una prueba autenticada de origen.

Métrica	**Significado exacto**
leidos	Registros no vacíos extraídos completamente. Una cola no leída tras corrupción no se cuenta.
procesados	Insertados confirmados en MongoDB + duplicados reconocidos. No implica éxito en Neo4j.
rechazados	Rechazos de validación y escrituras MongoDB identificadas como fallidas.
duplicados	Subconjunto de procesados omitido por el índice único.
pendientes	Máximo de 0 y leidos - procesados - rechazados; resultado de persistencia no confirmado.
sincronizadosNeo4j	Filas cuyo lote Neo4j terminó correctamente; no es conteo de nodos creados.
duracionMs	Tiempo transcurrido por archivo y carga; durante ejecución sigue aumentando.

Estados: EN_PROCESO, EXITOSA, EXITOSA_CON_RECHAZOS y ERROR. Cualquier archivo con ERROR hace fallar el estado global, aunque otros terminaran. Sin transacción distribuida, una caída de Neo4j después de MongoDB deja datos parciales; reenviar el archivo cuando las bases estén disponibles omite documentos duplicados y reintenta MERGE. Un fallo de write concern tiene resultado incierto, no se inventa éxito ni rollback. Los detalles en memoria están acotados a 100 por archivo; la auditoría persistida depende de disponibilidad de MongoDB.

10. Modelo de persistencia

Colección MongoDB	Contenido y uso

evaluaciones	Fuente documental plana para estadísticas e historial. Índice hash_fila único sparse e índices por correlativo, materia/aprobado, carrera/año, geografía, tipo, edad y sexo/tipo.
aspirantes	Proyección embebida por correlativo. Ordena por año, oportunidad, fecha e _id; agrupa evaluaciones completas con $$ROOT y reemplaza mediante $out.
resumen_carrera	Vista materializada que se genera explícitamente mediante la consulta 6.
cargas	Estado, timestamps, duración, métricas y errores de cargas.
registros_rechazados	Carga, archivo, fila y causa. Índice carga_id para auditoría paginada.
consultas_personalizadas	Biblioteca de definiciones para MongoDB y Neo4j.

Neo4j: nodos Aspirante(correlativo), Carrera(nombre), Materia(nombre), Evaluacion(key), Departamento(nombre), Municipio(key) e Institucion(key). Restricciones únicas en cada identificador. Relaciones: ASPIRA_A, REALIZO, DE_MATERIA, PARA_CARRERA, PROCEDE_DE, UBICADA_EN y PERTENECE_A. MERGE evita duplicación al reintentar.
La clave del nodo Evaluacion combina correlativo, materia, año y oportunidad; MongoDB deduplica por hash de todos los campos normalizados. Son granularidades distintas: cambios en resultado o carrera del mismo intento pueden coexistir como documentos y actualizar/agregar información del mismo nodo. Este punto requiere política de corrección si se incorporan fuentes contradictorias. Institucion no representa un establecimiento individual identificado: el dataset solo permite una agrupación tipo/municipio/departamento.
No son entidades JPA ni repositorios ORM generados: la implementación usa Document, Map y agregaciones en MongoTemplate, y Cypher directo con el driver Neo4j. C14 cuenta oportunidades observadas distintas por año/persona/materia; C15 conserva historial completo; C20 devuelve porcentajes de aprobación y no aprobación.

11. Inventario del backend: clases y métodos

El inventario siguiente se extrajo de todos los archivos Java bajo src/main/java/com/db2/proyectofinaldb2. Se incluyen constructores, métodos públicos y privados y referencias a la línea de declaración. Las firmas se simplifican quitando anotaciones de parámetros; el archivo fuente conserva el contrato exacto. Los métodos de records generados por Java (accesores, equals, hashCode y toString) se describen como implícitos, no como código escrito a mano.

ProyectoFinalDb2Application

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/ProyectoFinalDb2Application.java
Punto de entrada SpringApplication.run; activa la autoconfiguración y el escaneo del paquete raíz.

Método / firma y línea	Responsabilidad
L9: public static void main(String[] args)	Arranca Spring Boot y su contexto de aplicación.

AsyncConfig

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/config/AsyncConfig.java
Define el bean ExecutorService de dos hilos daemon para cargas y su shutdown al cerrar Spring.

Método / firma y línea	Responsabilidad
L16: public ExecutorService etlExecutor()	Crea el pool de dos hilos dedicado a ETL; no persiste tareas.

DotenvEnvironmentPostProcessor

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/config/DotenvEnvironmentPostProcessor.java
Carga dotenv antes de los beans, respetando prioridad de entorno/CLI y nombres relajados de Spring.

Método / firma y línea	Responsabilidad
L19: public void postProcessEnvironment(ConfigurableEnvironment environment, SpringApplication application)	Selecciona el directorio dotenv y llama a load antes de crear beans.
L27: static void load(Path directory, ConfigurableEnvironment environment)	Lee dotenv e inserta propiedades después de las variables del sistema.
L40: public int getOrder()	Devuelve prioridad máxima de ejecución del postprocesador.

MongoIndexInitializer

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/config/MongoIndexInitializer.java
ApplicationRunner que intenta crear índices MongoDB al arrancar.

Referencias/estado principales: EvaluationRepository repository.

Método / firma y línea	Responsabilidad
L20: public MongoIndexInitializer(EvaluationRepository repository)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L25: public void run(ApplicationArguments args)	Al iniciar Spring, invoca ensureIndexes; registra fallos de conexión para diagnóstico.

Neo4jConfig

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/config/Neo4jConfig.java
Bean Driver de respaldo cuando la autoconfiguración no proporciona uno; autentica con valores privados.

Método / firma y línea	Responsabilidad
L20: public Driver neo4jDriver(String uri, String username, String password)	Crea Driver mediante URI y AuthTokens.basic, sin credenciales por defecto.

Neo4jSchemaInitializer

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/config/Neo4jSchemaInitializer.java
ApplicationRunner que crea restricciones e índices Neo4j de forma idempotente.
Referencias/estado principales: Driver driver.

Método / firma y línea	Responsabilidad
L22: public Neo4jSchemaInitializer(Driver driver)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L27: public void run(ApplicationArguments args)	Al iniciar Spring, ejecuta restricciones e índices del catálogo; registra fallos de conexión.

WebConfig

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/config/WebConfig.java
Implementa WebMvcConfigurer para configurar los orígenes CORS autorizados.

Método / firma y línea	Responsabilidad
L14: public WebConfig(String allowedOrigins)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.

L22: public void addCorsMappings(CorsRegistry registry)	Registra reglas CORS para la API según orígenes configurados.

BusquedaController

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/controller/BusquedaController.java
Adaptador REST de búsqueda, ficha de aspirante e historial; el término vacío devuelve un listado general acotado.
Referencias/estado principales: SearchService searchService.

Método / firma y línea	**Responsabilidad**
L18: public BusquedaController(SearchService searchService)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L23: public List<Map<String, Object>> search(String tipo, String query)	Despacha búsqueda por tipo, valida criterio y delega filtros documentales con texto escapado.
L29: public Map<String, Object> aspirant(String correlativo)	Devuelve la ficha completa del correlativo solicitado.
L34: public List<Map<String, Object>> history(String correlativo)	Devuelve historial correspondiente al recurso del controlador.

CargaController

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/controller/CargaController.java
Adaptador REST de archivos multipart, historial, reporte y rechazos de cargas.
Referencias/estado principales: CargaService cargaService.

Método / firma y línea	**Responsabilidad**
L24: public CargaController(CargaService cargaService)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L29: public ResponseEntity<CargaReporteDto> upload(MultipartFile[] files)	Recibe files multipart y responde 202 con el reporte inicial.
L35: public List<Map<String, Object>> history()	Devuelve historial correspondiente al recurso del controlador.
L40: public Object detail(String id)	Obtiene estado y métricas de una carga por ID.
L45: public List<Map<String, Object>> rejected(String id, int page, int size)	Devuelve rechazos de una carga aplicando paginación.

CatalogoController

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/controller/CatalogoController.java
Expone catálogos derivados de los datos para filtros de la UI.
Referencias/estado principales: CatalogService catalogService.

Método / firma y línea	**Responsabilidad**
L17: public CatalogoController(CatalogService catalogService)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L22: public List<Map<String, Object>> get(String tipo)	Expone catálogo solicitado mediante CatalogService.

ConsultaMongoController

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/controller/ConsultaMongoController.java
Expone consultas analíticas numeradas, la consulta ejecutada, el resumen de carreras y la ejecución editada de solo lectura.
Referencias/estado principales: MongoQueryService queryService; ConsultaEditorService editorService.

Método / firma y línea	**Responsabilidad**
L20: public ConsultaMongoController(MongoQueryService queryService)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L25: public List<Map<String, Object>> execute(int numero, Integer anio, String materia, String carrera, String depto, String tipo, Integer limit, String correlativo)	Adapta parámetros HTTP y delega la ejecución al servicio correspondiente; no construye consultas desde la UI.
L38: public List<Map<String, Object>> generateCareerSummary()	Ejecuta consulta 6 y reconstruye resumen_carrera.

GrafoController

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/controller/GrafoController.java
Expone consultas Cypher con su sentencia y parámetros, visualización, detalle de nodos y ejecución editada de solo lectura.

Referencias/estado principales: GraphService graphService; ConsultaEditorService editorService.

Método / firma y línea	**Responsabilidad**
L19: public GrafoController(GraphService graphService)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L24: public List<Map<String, Object>> execute(int numero, String carrera, String depto, String materia, String tipo, Integer anio, Integer limit)	Adapta parámetros HTTP y delega la ejecución al servicio correspondiente; no construye consultas desde la UI.
L35: public GrafoDto visualization(String carrera, String depto, String materia, String tipo, Integer anio, Integer limit)	Construye/retorna GrafoDto para exploración visual con filtros.
L45: public Map<String, Object> node(String id)	Delega detalle de un nodo por elementId.

HealthController

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/controller/HealthController.java
Comprueba ambas bases de forma independiente y agrega conteos documentales.
Referencias/estado principales: MongoTemplate mongoTemplate; Driver driver; EvaluationRepository repository.

Método / firma y línea	Responsabilidad
L22: public HealthController(MongoTemplate mongoTemplate, Driver driver, EvaluationRepository repository)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L29: public Map<String, Object> health()	Hace ping MongoDB y Neo4j, captura fallos independientes y devuelve estado global y conteos.

SavedQueryController

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/controller/SavedQueryController.java
Verifica cabecera administrativa en todas sus operaciones y delega biblioteca de consultas.
Referencias/estado principales: SavedQueryService service.

Método / firma y línea	**Responsabilidad**

L24: public SavedQueryController(SavedQueryService service)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L27: public List<Map<String, Object>> list(String key, String engine)	Lista definiciones persistidas del motor autorizado.
L35: public Map<String, Object> save(String key, SavedQueryRequest request)	Valida motor/código, compila Cypher si aplica y persiste nueva definición con metadatos.
L42: public Map<String, Object> execute(String key, String id, boolean confirmWrite)	Adapta parámetros HTTP y delega la ejecución al servicio correspondiente; no construye consultas desde la UI.

ApiError

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/dto/ApiError.java
Record del error HTTP uniforme, sin lógica de persistencia.
Record ApiError: String timestamp, int status, String error, String mensaje, String ruta. Constructor canónico y accesores implícitos de Java.

CargaReporteDto

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/dto/CargaReporteDto.java
Record de reporte de carga con records anidados de totales, archivo y error.
Record CargaReporteDto: String id, String estado, String fechaInicio, String fechaFin, long duracionMs, int progreso, TotalesDto totales, List<ArchivoReporteDto> archivos, String error. Constructor canónico y accesores implícitos de Java.
Record TotalesDto: int leidos, int procesados, int rechazados, int duplicados, int pendientes, int sincronizadosNeo4j. Constructor canónico y accesores implícitos de Java.
Record ArchivoReporteDto: String nombre, int registrosLeidos, int registrosProcesados, int registrosRechazados, int duplicadosOmitidos, long duracionMs, String estado, List<ErrorRegistroDto> errores, int registrosPendientes, int sincronizadosNeo4j, String detalleError. Constructor canónico y accesores implícitos de Java.
Record ErrorRegistroDto: int fila, String causa, String valor. Constructor canónico y accesores implícitos de Java.

CustomQueryRequest

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/dto/CustomQueryRequest.java
Record de entrada para ejecutar una versión editada de una consulta; no guarda estado ni credenciales.
Record CustomQueryRequest: String codigo, Map<String, Object> parametros. Constructor canónico y accesores implícitos de Java.

GrafoDto

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/dto/GrafoDto.java
Record serializable con listas de nodos y relaciones; adapta el formato esperado por el cliente.
Record GrafoDto: List<Nodo> nodes, List<Relacion> links. Constructor canónico y accesores implícitos de Java.
Record Nodo: String id, String label, String tipo, Map<String, Object> propiedades. Constructor canónico y accesores implícitos de Java.
Record Relacion: String id, String source, String target, String tipo, Map<String, Object> propiedades. Constructor canónico y accesores implícitos de Java.

QueryResult

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/dto/QueryResult.java
Resultado de una consulta analítica: las filas y la consulta ejecutada que se muestra al usuario.
Record QueryResult: List<Map<String, Object>> filas, QueryView consulta. Constructor canónico y accesores implícitos de Java.

QueryView

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/dto/QueryView.java
Contrato de la consulta ejecutada, editable en el cliente; conserva los parámetros Cypher nulos de forma explícita.
Record QueryView: String motor, String codigo, String nota, Map<String, Object> parametros. Constructor canónico y accesores implícitos de Java.
La anotación JsonInclude(ALWAYS) sobre parametros evita que se omitan claves con valor null, necesarias para reenviar la misma sentencia sin que Neo4j falle por parámetros faltantes.

SavedQueryRequest

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/dto/SavedQueryRequest.java
Record de entrada JSON con validación Jakarta: nombre, descripción, motor y código.
Record SavedQueryRequest: String name, String description, String engine, String code. Constructor canónico y accesores implícitos de Java.

ApiException

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/exception/ApiException.java

Excepción de dominio HTTP que conserva el status y un mensaje controlado.
Referencias/estado principales: HttpStatus status.

Método / firma y línea	**Responsabilidad**
L9: public ApiException(HttpStatus status, String message)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L14: public HttpStatus getStatus()	Obtiene código HTTP asociado a ApiException.

GlobalExceptionHandler

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/exception/GlobalExceptionHandler.java
Advice REST que convierte excepciones de validación, protocolo, datos y drivers en ApiError.

Método / firma y línea	Responsabilidad
L28: public ResponseEntity<ApiError> handleApiException(ApiException exception, HttpServletRequest request)	Manejador central de excepciones de dominio HTTP. Construye respuesta uniforme y controlada.
L33: public ResponseEntity<ApiError> handleValidation(MethodArgumentNotValidException exception, HttpServletRequest request)	Manejador central de validación Jakarta. Construye respuesta uniforme y controlada.
L43: public ResponseEntity<ApiError> handleMissingParameter(Exception exception, HttpServletRequest request)	Manejador central de parámetros requeridos ausentes. Construye respuesta uniforme y controlada.
L48: public ResponseEntity<ApiError> handleTypeMismatch(MethodArgumentTypeMismatchException exception, HttpServletRequest request)	Manejador central de tipos de parámetros incorrectos. Construye respuesta uniforme y controlada.
L55: public ResponseEntity<ApiError> handleUnreadableBody(HttpMessageNotReadableException exception, HttpServletRequest request)	Manejador central de JSON/cuerpo ilegible. Construye respuesta uniforme y controlada.
L61: public ResponseEntity<ApiError> handleNotFound(NoResourceFoundException exception, HttpServletRequest request)	Manejador central de recursos o rutas inexistentes. Construye respuesta uniforme y controlada.
L67: public ResponseEntity<ApiError> handleMaxUploadSize(MaxUploadSizeExceededException exception, HttpServletRequest request)	Manejador central de multipart demasiado grande (413). Construye respuesta uniforme y controlada.

L73: public ResponseEntity<ApiError> handleMongo(MongoException exception, HttpServletRequest request)	Manejador central de fallos del driver MongoDB (503). Construye respuesta uniforme y controlada.
L80: public ResponseEntity<ApiError> handleNeo4j(Neo4jException exception, HttpServletRequest request)	Manejador central de fallos del driver Neo4j (503). Construye respuesta uniforme y controlada.
L87: public ResponseEntity<ApiError> handleUnexpected(Exception exception, HttpServletRequest request)	Manejador central de excepciones no previstas (500). Construye respuesta uniforme y controlada.
L93: private ResponseEntity<ApiError> build(HttpStatus status, String message, HttpServletRequest request)	Construye ResponseEntity con ApiError, status y ruta de petición.

CypherCatalog

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/neo4j/CypherCatalog.java
Catálogo de constantes Cypher: carga MERGE, restricciones, índices, siete consultas y visualización.
No declara métodos propios. Constantes: CONSTRAINTS, INDEXES, LOAD_BATCH, G1_SUBGRAPH_BY_CAREER, G2_CAREERS_BY_DEPARTMENT, G3_APPROVED_BY_SUBJECT, G4_CO_ASPIRANTS, G5_SHARED_SUBJECTS, G6_FULL_PATH, G7_SHORTEST_PATH y VISUALIZATION_BASE. No almacena credenciales.

Método / firma y línea	Responsabilidad
L9: private CypherCatalog()	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.

EvaluationRepository

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/repository/EvaluationRepository.java
Única fachada documental compartida del ETL, consultas, catálogos y búsquedas; MongoTemplate y driver MongoDB.
Referencias/estado principales: MongoTemplate mongoTemplate.
Record InsertResult: int inserted, int duplicates, int failed, List<Integer> failedIndexes.
Constructor canónico y accesores implícitos de Java.

Método / firma y línea	Responsabilidad
L32: public EvaluationRepository(MongoTemplate mongoTemplate)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.

L36: public void ensureIndexes()	Crea índices analíticos y único sparse por hash_fila; índice de auditoría por carga_id.
L52: public InsertResult insertBatch(List<Document> documents)	Inserta unordered, clasifica duplicados y errores por índice; usa confirmación del driver y propaga write concern incierto.
L81: public void insertRejected(List<Document> documents)	Persiste documentos de auditoría en lotes.
L93: public void rebuildAspirantes()	Ordena registros, agrupa por correlativo con $$ROOT y reemplaza colección derivada mediante $out.
L114: public List<Document> aggregate(String collection, List<Document> pipeline)	Ejecuta pipeline de la colección indicada y materializa el resultado en una lista.
L120: public Document findAspirante(String correlativo)	Busca documento embebido por _id textual.
L126: public List<Document> findEvaluationsByAspirante(String correlativo)	Consulta evaluaciones planas y ordena por año, oportunidad y fecha.
L132: public List<Document> findRejected(String cargaId, int page, int size)	Consulta auditoría por carga_id, ordena y pagina.
L140: public long count(String collection)	Devuelve conteo estimado de documentos.
L144: public void saveLoad(Document document)	Inserta reporte de carga.
L148: public void replaceLoad(Document document)	Actualiza/reemplaza reporte por _id con upsert.
L154: public List<Document> findLoads(int limit)	Lista reportes recientes por fecha descendente.
L162: public Document findLoad(String id)	Obtiene un reporte persistido por ID.
L167: public InsertResult(int inserted, int duplicates, int failed)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.

CargaJob

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/service/CargaJob.java
Estado mutable concurrente de una carga y de sus FileJob; snapshots para HTTP y persistencia.
Referencias/estado principales: String id; Instant fechaInicio; List<FileJob> archivos; String nombre; Path ruta; AtomicInteger leidos; AtomicInteger procesados; AtomicInteger rechazados; AtomicInteger duplicados; AtomicInteger sincronizadosNeo4j.
Tipos anidados: FileJob (estado por archivo y contadores atómicos) y Estado (EN_PROCESO, EXITOSA, EXITOSA_CON_RECHAZOS, ERROR). Los métodos duplicados snapshot/toDocument/getEstado pertenecen al job o a FileJob según su ubicación en el fuente.

Método / firma y línea	Responsabilidad
L31: public CargaJob(String id)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L35: public String getId()	Accesor: devuelve Id del estado del objeto; no inicia operaciones de red.
L39: public Estado getEstado()	Accesor: devuelve Estado del estado del objeto; no inicia operaciones de red.
L43: public Instant getFechaInicio()	Accesor: devuelve FechaInicio del estado del objeto; no inicia operaciones de red.
L47: public List<FileJob> getArchivos()	Accesor: devuelve Archivos del estado del objeto; no inicia operaciones de red.
L51: public void addFile(FileJob file)	Agrega FileJob a la lista concurrente de archivos.
L55: public void finish()	Deriva estado global, prioriza ERROR y registra fecha final.
L70: public void fail(String message)	Asigna mensaje controlado, ERROR y fecha final.
L76: public String getError()	Accesor: devuelve Error del estado del objeto; no inicia operaciones de red.
L80: public long getDuracionMs()	Calcula duración desde inicio hasta fin o instante actual.
L85: public int getProgreso()	Calcula progreso según archivos terminados; antes del fin no supera 99.
L96: public int totalLeidos()	Suma el contador Leidos de todos los FileJob de la carga.
L100: public int totalProcesados()	Suma el contador Procesados de todos los FileJob de la carga.
L104: public int totalRechazados()	Suma el contador Rechazados de todos los FileJob de la carga.
L108: public int totalDuplicados()	Suma el contador Duplicados de todos los FileJob de la carga.
L112: public CargaReporteDto snapshot()	Crea record DTO a partir de copias/contadores para consulta concurrente por HTTP.
L131: public Document toDocument()	Serializa el estado a Document para persistir el reporte.
L162: public AtomicInteger getSincronizadosNeo4j()	Accesor: devuelve SincronizadosNeo4j del estado del objeto; no inicia operaciones de red.
L164: public int pending()	Deriva filas extraídas sin confirmación MongoDB: max(0, leidos-procesados-rechazados).

L171: public FileJob(String nombre, Path ruta)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L176: public String getNombre()	Accesor: devuelve Nombre del estado del objeto; no inicia operaciones de red.
L180: public Path getRuta()	Accesor: devuelve Ruta del estado del objeto; no inicia operaciones de red.
L184: public AtomicInteger getLeidos()	Accesor: devuelve Leidos del estado del objeto; no inicia operaciones de red.
L188: public AtomicInteger getProcesados()	Accesor: devuelve Procesados del estado del objeto; no inicia operaciones de red.
L192: public AtomicInteger getRechazados()	Accesor: devuelve Rechazados del estado del objeto; no inicia operaciones de red.
L196: public AtomicInteger getDuplicados()	Accesor: devuelve Duplicados del estado del objeto; no inicia operaciones de red.
L200: public Estado getEstado()	Accesor: devuelve Estado del estado del objeto; no inicia operaciones de red.
L204: public void setEstado(Estado estado)	Actualiza Estado del estado mutable del archivo de carga.
L208: public void setDuracionMs(long duracionMs)	Actualiza DuracionMs del estado mutable del archivo de carga.
L212: public void setDetalleError(String detalleError)	Actualiza DetalleError del estado mutable del archivo de carga.
L216: public void addError(int fila, String causa, String valor)	Agrega detalle solo mientras haya menos de 100 incidencias.
L222: public CargaReporteDto.ArchivoReporteDto snapshot()	Crea record DTO a partir de copias/contadores para consulta concurrente por HTTP.
L242: public Document toDocument()	Serializa el estado a Document para persistir el reporte.

CargaService

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/service/CargaService.java
Coordina recepción, temporales, jobs en memoria, executor y registro del reporte.
Referencias/estado principales: EvaluationRepository repository; EtlService etlService; ExecutorService etlExecutor; Map<String, CargaJob> jobs; String configuredTempDir.

Método / firma y línea	Responsabilidad
L37: public CargaService(EvaluationRepository repository, EtlService etlService,	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.

ExecutorService etlExecutor, String configuredTempDir)	
L47: public CargaReporteDto start(MultipartFile[] files)	Valida multipart, copia temporales, registra job, verifica índices, programa ETL y devuelve snapshot. Gestiona fallos de programación y limpieza.
L129: public Object getJob(String id)	Prefiere job en memoria; si no existe consulta cargas y normaliza BSON.
L141: public List<Map<String, Object>> getJobs()	Lista hasta 50 reportes persistidos.
L145: public List<Map<String, Object>> getRejected(String cargaId, int page, int size)	Valida page, acota size a 1..500 y consulta rechazos.
L153: private String sanitize(String name)	Sustituye caracteres no permitidos en el nombre usado dentro del directorio temporal.
L158: private void cleanup(Path directory)	Recorre únicamente el directorio temporal de esa carga y elimina sus archivos en orden inverso; best effort.

CatalogService

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/service/CatalogService.java
Construye agregaciones para opciones de catálogos y sus conteos.
Referencias/estado principales: EvaluationRepository repository.

ConsultaEditorService

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/service/ConsultaEditorService.java
Ejecuta versiones editadas de consultas con política de solo lectura, límite de filas y timeout.
Referencias/estado principales: MongoTemplate mongo; Driver driver; CustomQueryPolicy policy; int maxRows; int timeout.
Métodos principales: executeMongo(String codigo): Map y executeNeo4j(String codigo, Map<String, Object> parametros): Map. Ambos devuelven filas, truncado, limiteFilas y duracionMs; las escrituras, colecciones no permitidas y errores de sintaxis se rechazan con ApiException.

Método / firma y línea	Responsabilidad
L18: public CatalogService(EvaluationRepository repository)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.

L22: public List<Map<String, Object>> get(String type)	Selecciona catálogo permitido y ejecuta agregación con conteos de aspirantes/filas según catálogo.
L66: private Document groupWithSet(String field)	Agrupa por campo y acumula correlativos distintos para conteo de aspirantes.
L71: private Document projectValue(String valueExpression, String labelExpression, String extraField)	Proyecta value, label, count y campo extra cuando corresponde.
L82: private Document sort(String field, int direction)	Construye etapa $sort para el pipeline.

CustomQueryPolicy

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/service/CustomQueryPolicy.java
Valida motor, operaciones, colecciones y sintaxis permitida de consultas administrativas.

Método / firma y línea	Responsabilidad
L22: public String engine(String value)	Normaliza y restringe motor a MONGODB o NEO4J.
L30: public boolean writes(String engine, String code)	Valida longitud y código; detecta si la consulta puede escribir.
L46: public Document mongo(String code)	Parsea JSON y comprueba operación, filtro, pipeline y estructura según política.
L85: private void collection(Object name)	Exige colección dentro de la lista permitida.
L91: private void validateTree(Object value)	Recorre árboles JSON y bloquea JavaScript, acceso al servidor y uniones con colecciones no permitidas.
L109: private boolean containsWriteStage(Object value)	Detecta $out/$merge de forma recursiva, incluso dentro de estructuras anidadas.
L117: private String cypherTokens(String code)	Elimina literales y comentarios al inspeccionar tokens ejecutables; detecta cierres inválidos.
L157: private ApiException invalid(String message)	Construye ApiException de petición inválida con mensaje controlado.

EtlService

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/service/EtlService.java
Orquesta lectura secuencial, transformación, lotes, rechazo, grafo y reconstrucción documental.
Referencias/estado principales: EvaluationRepository repository; TransformationService transformationService; Driver driver; int batchSize.

Método / firma y línea	Responsabilidad

L32: public EtlService(EvaluationRepository repository, TransformationService transformationService, Driver driver, int batchSize)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L45: public void process(CargaJob job)	Recorre archivos, acumula estado, controla excepciones y reconstruye aspirantes si hubo confirmaciones MongoDB.
L80: private int processFile(CargaJob job, CargaJob.FileJob file)	Valida encabezado y cantidad de columnas; lee filas, transforma y vacía lotes de válidos o rechazos.
L137: private void flush(CargaJob job, CargaJob.FileJob file, List<Document> mongoBatch, List<Map<String, Object>> graphBatch, List<Document> rejectedBatch)	Persiste rechazos y lote MongoDB, actualiza métricas, excluye errores conocidos y carga el lote aceptado en Neo4j.
L181: private void loadGraph(List<Map<String, Object>> rows)	Abre sesión Neo4j y ejecuta LOAD_BATCH en una transacción local de escritura.
L193: private Document rejectedDocument(CargaJob job, CargaJob.FileJob file, int line, String cause, String[] row)	Construye documento de auditoría con carga, archivo, fila, causa, muestra acotada y fecha.

GraphService

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/service/GraphService.java
Lee Cypher con filtros, transforma nodos y relaciones al DTO de grafo y expone la sentencia con sus parámetros en runQueryWithPlan.
Referencias/estado principales: Driver driver.

Método / firma y línea	Responsabilidad
L27: public GraphService(Driver driver)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L31: public List<Map<String, Object>> runQuery(int number, String carrera, String depto, String materia, String tipo, Integer anio, Integer limit)	Selecciona una de siete consultas, valida G7, prepara parámetros y ejecuta lectura Neo4j.
L73: public GrafoDto visualization(String carrera, String depto, String materia, String tipo, Integer anio, Integer limit)	Construye/retorna GrafoDto para exploración visual con filtros.
L145: public Map<String, Object> nodeDetail(String id)	Recupera nodo y vecinos por elementId y normaliza sus propiedades.
L161: private void addNode(Map<String, GrafoDto.Nodo> nodes, Node node)	Deduplica nodos del resultado por identificador en el mapa de visualización.

L174: private void addLink(Map<String, GrafoDto.Relacion> links, Node source, String type, Node target)	Deduplica relaciones del resultado por identificador.
L183: private Node node(Record record, String key)	Delega detalle de un nodo por elementId.
L192: private Object normalize(Object value)	Normaliza recursivamente valores BSON/temporales o valores del driver de grafo a objetos serializables, según esta clase.
L221: private Map<String, Object> nodeToMap(Node node)	Convierte propiedades/etiquetas de Node a un mapa serializable.
L235: private Map<String, Object> relationshipToMap(Relationship relationship)	Convierte Relationship con extremos y tipo a mapa JSON.
L247: private String blankToNull(String value)	Convierte texto vacío a null para filtros opcionales.
L251: private boolean notBlank(String value)	Comprueba si hay un filtro textual no vacío.

MongoQueryService

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/service/MongoQueryService.java
Implementa las veinte consultas analíticas mediante pipelines y filtros compartidos; executeWithPlan devuelve filas y la consulta ejecutada.
Referencias/estado principales: EvaluationRepository repository.
Record QueryFilters: Integer anio, String materia, String carrera, String depto, String tipo, Integer limit, String correlativo. Constructor canónico y accesores implícitos de Java.
La consulta visible se representa como find en C11 y C15, como aggregate en el resto y con una nota en C6 porque la lectura de resumen_carrera se regenera con $merge.

Método / firma y línea	Responsabilidad
L18: public MongoQueryService(EvaluationRepository repository)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L22: public List<Map<String, Object>> execute(int number, QueryFilters filters)	Despacha C1..C20, valida número y correlativo cuando corresponde, y normaliza filas BSON.
L50: private List<Map<String, Object>> aspirantsByInstitutionType(QueryFilters filters)	C01. Aspirantes distintos por tipo de institución. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L67: private List<Map<String, Object>> approvedBySubject(QueryFilters filters)	C02. Aprobados por materia. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.

L83: private List<Map<String, Object>> approvedByCareerAndYear(QueryFilters filters)	C03. Aprobados por carrera y año. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L107: private List<Map<String, Object>> approvalRateBySubject(QueryFilters filters)	C04. Tasa de aprobación por materia. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L131: private List<Map<String, Object>> averageAgeByCareer(QueryFilters filters)	C05. Edad promedio por carrera. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L162: private List<Map<String, Object>> careerSummary()	C06. Resumen materializado por carrera. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L225: private List<Map<String, Object>> averageAgeApprovedByCareerAndType(QueryFilters filters)	C07. Edad promedio de aprobados por carrera y tipo institucional. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L258: private List<Map<String, Object>> approvedByMunicipalityAndCareer(QueryFilters filters)	C08. Aprobados por municipio y carrera. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L291: private List<Map<String, Object>> evaluationsByMonthAndSubject(QueryFilters filters)	C09. Evaluaciones por mes y materia. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L311: private List<Map<String, Object>> topCareersByAge(QueryFilters filters)	C10. Carreras con más aspirantes por edad. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L331: private List<Map<String, Object>> performanceHistory(QueryFilters filters)	C11. Historial de desempeño del aspirante. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L337: private List<Map<String, Object>> distributionBySexAndType(QueryFilters filters)	C12. Distribución por sexo y tipo institucional. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L359: private List<Map<String, Object>> approvalRateByAge(QueryFilters filters)	C13. Tasa de aprobación por edad. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L385: private List<Map<String, Object>> averageAttemptsBySubject(QueryFilters filters)	C14. Intentos observados distintos por persona, materia y año; promedio por materia. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L420: private List<Map<String, Object>> fullAspirantHistory(QueryFilters filters)	C15. Ficha completa con documento e historial embebido. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.

L431: private List<Map<String, Object>> careersWithMostFirstAttemptFailures(Query Filters filters)	C16. Carreras con más fallos de primera oportunidad. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L452: private List<Map<String, Object>> topMunicipalities(QueryFilters filters)	C17. Municipios con mayor participación. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L475: private List<Map<String, Object>> approvalRateByInstitutionType(QueryFilters filters)	C18. Aprobación por tipo institucional. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L499: private List<Map<String, Object>> topCareersByDepartment(QueryFilters filters)	C19. Carreras más solicitadas por departamento. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L539: private List<Map<String, Object>> approvalEvolutionByYear(QueryFilters filters)	C20. Evolución anual con porcentajes de aprobación y no aprobación. Construye y ejecuta la agregación correspondiente respetando filtros aplicables.
L565: private List<Map<String, Object>> run(List<Document> pipeline)	Ejecuta pipeline o inicializador según clase; ver responsabilidad específica del archivo.
L569: private void addMatch(List<Document> pipeline, QueryFilters filters)	Añade filtros globales a la lista de etapas cuando hay condiciones.
L576: private Document baseMatch(QueryFilters filters)	Construye condiciones comunes de año, materia, carrera, departamento y tipo.
L598: private void requireCorrelativo(QueryFilters filters)	Exige correlativo no vacío en consultas de historial/ficha.
L605: private int limit(QueryFilters filters, int defaultValue)	Aplica tope de resultados según configuración de filtros.
L612: private boolean notBlank(String value)	Comprueba si hay un filtro textual no vacío.
L616: private Document sort(String field, int direction)	Construye etapa $sort para el pipeline.
L620: private Document parse(String json)	Convierte una definición JSON estática en Document.

SavedQueryService

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/service/SavedQueryService.java
Autoriza, guarda y ejecuta definiciones; MongoTemplate directo para la biblioteca y Driver para Cypher.

Referencias/estado principales: MongoTemplate mongo; Driver driver; CustomQueryPolicy policy; String adminKey; int maxRows; int timeout.

Método / firma y línea	Responsabilidad
L47: public SavedQueryService(MongoTemplate mongo, Driver driver, CustomQueryPolicy policy, String adminKey, int maxRows, int timeout)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L59: public void authorize(String key)	Compara clave recibida con configuración privada; falla si falta o no coincide.
L69: public List<Map<String, Object>> list(String engine)	Lista definiciones persistidas del motor autorizado.
L75: public Map<String, Object> save(SavedQueryRequest request)	Valida motor/código, compila Cypher si aplica y persiste nueva definición con metadatos.
L88: public Map<String, Object> execute(String id, boolean confirmed)	Recupera definición, revalida política, exige confirmación si escribe, ejecuta motor y devuelve resultado acotado.
L126: private Map<String, Object> executeMongo(String code)	Ejecuta operación JSON permitida con límites; devuelve resultados o métricas de escritura.
L158: private Map<String, Object> executeNeo4j(String code)	Ejecuta Cypher validado con timeout/límite y normalización de resultados.
L181: private boolean compileNeo4j(String code)	Usa EXPLAIN para validar consulta y determinar modo de escritura.
L194: private Map<String, Object> response(List<Map<String, Object>> rows, Map<String, Object> changes, boolean truncated)	Arma respuesta con resultados, métricas y truncamiento de presentación.
L202: private Object graphValue(Object value)	Normaliza valores del driver para resultados administrativos JSON.

SearchService

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/service/SearchService.java
Búsqueda por tipos, fichas e historial; el término vacío lista hasta 50 aspirantes ordenados por identificador.
Referencias/estado principales: MongoTemplate mongoTemplate; EvaluationRepository repository.

Método / firma y línea	Responsabilidad
L22: public SearchService(MongoTemplate mongoTemplate, EvaluationRepository repository)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.

L27: public List<Map<String, Object>> search(String type, String text)	Despacha búsqueda por tipo, valida criterio y delega filtros documentales con texto escapado.
L47: public Map<String, Object> getAspirant(String correlativo)	Busca ficha o devuelve recurso no encontrado.
L56: public List<Map<String, Object>> getHistory(String correlativo)	Retorna historial plano ordenado por correlativo.
L60: private List<Map<String, Object>> searchAspirants(String text)	Construye agregación específica de búsqueda de aspirantes.
L69: private List<Map<String, Object>> searchCareers(String text)	Construye agregación específica de búsqueda de carreras.
L83: private List<Map<String, Object>> searchSubjects(String text)	Construye agregación específica de búsqueda de materias.
L97: private List<Map<String, Object>> searchInstitutions(String text)	Construye agregación específica de búsqueda de instituciones agrupadas.
L120: private List<Map<String, Object>> searchMunicipalities(String text)	Construye agregación específica de búsqueda de municipios.
L138: private List<Map<String, Object>> searchDepartments(String text)	Construye agregación específica de búsqueda de departamentos.
L152: private List<Map<String, Object>> run(List<Document> pipeline)	Ejecuta pipeline o inicializador según clase; ver responsabilidad específica del archivo.
L157: private Document regex(String text)	Escapa texto con Pattern.quote para búsqueda literal case-insensitive.

TransformationService

Archivo:
proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/service/TransformationService.java
Reglas autoritativas de validación, normalización, edad, hash y claves de grafo.
Record TransformedRow: Document mongo, Map<String, Object> graph, int line. Constructor canónico y accesores implícitos de Java.
Tipos anidados: TransformedRow, resultado para ambos motores; RowValidationException, rechazo controlado de una fila.

Método / firma y línea	**Responsabilidad**
L44: public Map<String, Integer> mapHeaders(String[] headers)	Normaliza nombres/alias y rechaza encabezados semánticamente duplicados.
L57: public TransformedRow transform(String[] row, Map<String, Integer> indexes, String fileName, int line)	Valida los mínimos y límites; produce documento MongoDB y parámetros de grafo, recalculando hash y derivados.
L166: public String safeRow(String[] row)	Concatena una muestra del registro acotada a 300 caracteres para auditoría.

L171: private String value(String[] row, Map<String, Integer> indexes, String field)	Obtiene una celda por índice canónico o devuelve vacío si falta.
L179: private String clean(String value)	Elimina BOM y colapsa espacios Unicode.
L183: private String defaultIfBlank(String value, String fallback)	Aplica valor de respaldo solamente si el texto está vacío.
L187: private Integer parseInt(String value)	Convierte entero estricto o devuelve null si no se reconoce.
L199: private Integer parseYear(String value)	Valida nacimiento en 1900..año actual; fuera de rango retorna null.
L208: private LocalDate parseDate(String value)	Prueba formatos con calendario estricto; devuelve null si no reconoce la fecha.
L223: private String normalizeSex(String value)	Transforma alias a M/F o DESCONOCIDO.
L234: private String normalizeInstitutionType(String value)	Reconoce PUBLICO/PRIVADO y equivalentes o conserva categoría normalizada.
L249: private String stripAccents(String value)	Descompone Unicode y elimina marcas diacríticas.
L254: private String normalizeHeader(String header)	Normaliza tildes, BOM, mayúsculas y separadores a nombre canónico.
L263: private static Map<String, String> buildAliases()	Construye catálogo cerrado de alias reconocidos.
L295: private static void register(Map<String, String> aliases, String canonical, String... variants)	Añade variantes de encabezado a su campo canónico.
L299: private String hash(Object... values)	Calcula SHA-256 sobre los campos normalizados, en orden estable.
L311: private String safe(String value)	Acota texto individual usado en mensajes de error.
L322: public RowValidationException(String message)	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.

CsvSupport

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/util/CsvSupport.java
Utilidad estática de delimitadores y lectores UTF-8 estrictos OpenCSV.

Método / firma y línea	**Responsabilidad**
L20: private CsvSupport()	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.

L23: public static char detectDelimiter(Path path)	Inspecciona el encabezado, cuenta separadores fuera de comillas y elige coma, punto y coma o tabulador.
L59: public static CSVReader reader(Path path, char delimiter)	Crea lector UTF-8 con REPORT ante bytes inválidos, parser OpenCSV con separador elegido y límite de 100 líneas por registro multilínea.

MongoJson

Archivo: proyecto-final-db2/src/main/java/com/db2/proyectofinaldb2/util/MongoJson.java
Normaliza documentos, listas y tipos BSON/temporales para respuesta JSON.

Método / firma y línea	**Responsabilidad**
L20: private MongoJson()	Constructor: establece estado inicial o recibe dependencias/configuración por inyección; no es un endpoint.
L23: public static Map<String, Object> toMap(Document document)	Normaliza Document a mapa serializable JSON.
L32: public static List<Map<String, Object>> toList(List<Document> documents)	Normaliza lista de documentos a filas JSON.
L40: public static Object normalize(Object value)	Normaliza recursivamente valores BSON/temporales o valores del driver de grafo a objetos serializables, según esta clase.

12. Inventario del frontend

Los componentes son funciones React. Las relaciones entre vistas se implementan con composición JSX, props y callbacks; estado local con useState, efectos con useEffect, referencias con useRef y caché de cálculos con useMemo/useCallback. No hay Redux ni una capa de acceso directo a bases desde JavaScript.

src/App.jsx

Declara rutas: / Dashboard, /carga, /consultas, /grafos, /buscador; /inicio redirige y * muestra Error404.

Dependencias importadas: react-router-dom, ./components/Layout, ./pages/Dashboard, ./pages/Carga, ./pages/Consultas, ./pages/ExploradorGrafos, ./pages/Buscador, ./pages/Error404.

src/api/client.js

Axios con base pública VITE_API_URL o /api, timeout 180000 ms e interceptor de errores con mensaje y status.
Dependencias importadas: axios.

src/components/Cargando.jsx

Indicador reutilizable de espera con mensaje.

Función exportada	Parámetros (resumen)
Cargando	{ mensaje = 'Cargando información…' }

src/components/ConsultaEjecutada.jsx

Visor y editor de la consulta ejecutada al final de cada gráfica o tabla. Ejecuta la versión editada solo en lectura y muestra su resultado en una tabla aparte.
Dependencias importadas: react, ../api/client, ./TablaResultados, ../utils/formato.
Estado y acciones: codigo, abierto, ejecutando, error y resultado; ejecutar valida el JSON en MongoDB, restaurar recupera la sentencia original y el resultado editado no reemplaza la visualización original.

src/components/ConsultasGuardadas.jsx

Editor de nombre/descripcion/código por motor; clave solo en memoria; guardar, seleccionar, cancelar borrador y confirmar escritura con EJECUTAR.

Función exportada	Parámetros (resumen)
ConsultasGuardadas	{ motor }

Dependencias importadas: react, ../api/client, ./TablaResultados.

src/components/EstadoError.jsx

Muestra error legible y callback de reintento cuando corresponde.

Función exportada	Parámetros (resumen)
EstadoError	{ mensaje, onReintentar, titulo = 'No se pudo cargar la información' }

src/components/FichaAspirante.jsx

Ficha común entre C15 y buscador, resumen, historial completo y documento técnico JSON.

Función exportada	Parámetros (resumen)
FichaAspirante	{ ficha, historial }

Dependencias importadas: ./TablaResultados, ../utils/formato.

src/components/FiltroBar.jsx

Contenedor de filtros y acciones; exporta SelectCampo para select controlado con opciones.

Función exportada	Parámetros (resumen)
SelectCampo	{ etiqueta, valor, onChange, opciones = [], placeholder = 'Todos', disabled = false }
FiltroBar	{ children, onLimpiar, onAplicar, aplicando = false, textoAplicar = 'Aplicar filtros' }

src/components/Grafica.jsx

Wrapper de Recharts para series y tipos de gráfica configurados por cada consulta.

Función exportada	Parámetros (resumen)
Grafica	{ tipo = 'bar', datos = [], xKey = 'label', yKey = 'value', series = [], altura = 300, unidad = '', nombreY }

Dependencias importadas: recharts, ../utils/formato.

src/components/GrafoCanvas.jsx

Wrapper ForceGraph2D con tamaño, selección y colores/etiquetas por tipo; comunica clic de nodo mediante callback.

Función exportada	Parámetros (resumen)
GrafoCanvas	{ datos, onNodeClick, altura = 560 }

Dependencias importadas: react, react-force-graph-2d, ../utils/formato.

src/components/KpiCard.jsx

Presenta título, valor y detalle de un indicador.

Función exportada	Parámetros (resumen)
KpiCard	{ titulo, valor, detalle, tono = 'indigo', icono }

src/components/Layout.jsx

Contenedor común, navegación con NavLink/Outlet y comprobación de salud de la API.
Dependencias importadas: react, react-router-dom, ../api/client.

src/components/Modal.jsx

Diálogo accesible reutilizable con título, subtítulo y contenido; cierra con Escape, clic fuera o el botón de la cabecera.
Dependencias importadas: react.
Bloquea el scroll del documento mientras está abierto y restaura el valor previo al desmontar.

src/components/TablaResultados.jsx

Tabla reutilizable con paginación, inferencia/selección de columnas, formato y exportación CSV.

Función exportada	Parámetros (resumen)
TablaResultados	{ filas = [], columnas, titulo, exportar = true, paginada = true, maximo }

Dependencias importadas: react, ../utils/formato.

src/components/UploadDragDrop.jsx

Selector/drag-and-drop de CSV, lista de archivos y eliminación; extensión preliminar, estado ocupado y botón de revisión.

Función exportada	Parámetros (resumen)
UploadDragDrop	{ archivos, onArchivos, onQuitar, onSubir, cargando, progreso, error }

Dependencias importadas: react, ../utils/formato.

src/hooks/useCarga.js

Coordina multipart, polling del reporte, consulta de rechazados y reapertura de carga; limpia seguimiento al desmontar.
Dependencias importadas: react, ../api/client.

src/hooks/useCatalogos.js

Carga catálogos en paralelo mediante Promise.allSettled y conserva lo disponible ante fallos parciales.
Dependencias importadas: react, ../api/client.

src/hooks/useConsultas.js

Gestiona parámetros, carga/error y recarga de consulta; conserva la consulta ejecutada para el editor y permite regenerar el resumen C6.

Función exportada	Parámetros (resumen)
useConsultas	numero, filtros

Dependencias importadas: react, ../api/client.

src/main.jsx

Punto de entrada: createRoot, StrictMode y BrowserRouter; importa styles.css y monta App.
Dependencias importadas: react, react-dom/client, react-router-dom, ./App.

src/pages/Buscador.jsx

Busca por tipo y texto; q vacío lista hasta 50 resultados. Abre la ficha y el historial en un modal reutilizable.
Dependencias importadas: react, ../api/client, ../components/TablaResultados, ../components/Cargando, ../components/EstadoError, ../utils/formato, ../components/FichaAspirante, ../components/Modal.

src/pages/Carga.jsx

Selecciona archivos, invalida revisiones al cambiar selección, crea/termina Worker, muestra métricas previas y confirma envío de copias normalizadas. Luego seguimiento de jobs, historial y rechazos.
Dependencias importadas: react, ../api/client, ../components/UploadDragDrop, ../components/TablaResultados, ../components/Cargando, ../utils/formato, ../hooks/useCarga.
Métodos de coordinación: cambiarArchivos descarta revisión; iniciar valida lote y programa Worker; confirmarEnvio solo envía si todos los CSV están preparados; cargarHistorial, verRechazados y abrirHistorial consultan auditoría. Límites: 20 MB/archivo, 20 archivos y 100 MB/lote; el backend tiene límites multipart independientes.

src/pages/Consultas.jsx

Catálogo de 20 consultas, filtros, ejecución con hook, tabla/gráfica, editor de la consulta ejecutada y ficha completa en C15; biblioteca administrativa integrada.
Función exportada Parámetros (resumen)
Consultas

Dependencias importadas: react, ../components/Grafica, ../components/TablaResultados, ../components/Cargando, ../components/EstadoError, ../components/FiltroBar,

../hooks/useConsultas, ../hooks/useCatalogos, ../components/ConsultasGuardadas, ../components/ConsultaEjecutada, ../components/FichaAspirante, ../utils/formato.

src/pages/Dashboard.jsx

Carga en paralelo salud, catálogos y consultas analíticas; presenta KPI y gráficos.
Dependencias importadas: react, react-router-dom, ../api/client, ../components/KpiCard, ../components/Grafica, ../components/Cargando, ../components/EstadoError, ../utils/formato.

src/pages/Error404.jsx

Pantalla para rutas frontend no reconocidas; permite volver a inicio.
Dependencias importadas: react-router-dom.

src/pages/ExploradorGrafos.jsx

Visualización filtrada, consultas G1..G7 con su sentencia Cypher y parámetros, inspección de nodo y tabla; editor de la consulta ejecutada y consultas guardadas.
Dependencias importadas: react, ../api/client, ../components/GrafoCanvas, ../components/TablaResultados, ../components/Cargando, ../components/EstadoError, ../components/FiltroBar, ../hooks/useCatalogos, ../components/ConsultasGuardadas, ../components/ConsultaEjecutada, ../utils/formato.

src/utils/csvValidation.js

normalizeRecord aplica reglas locales; prepareCsv detecta separador desde el encabezado, valida estructura y prepara copia CSV; hasta 100 detalles visibles.

Función exportada	Parámetros (resumen)
normalizeRecord	record, year = new Date().getFullYear()
prepareCsv	text, name

Dependencias importadas: papaparse.
Rechazos de filas o estructura bloquean el lote completo en la UI, sin descarte silencioso. Avisos de nulos y NINGUNO no lo bloquean. Las filas se numeran como registros sin encabezado, no líneas físicas. Los originales no se modifican; el cliente envía nuevos File con CSV normalizado y BOM UTF-8.

src/utils/formato.js

Utilidades de números, porcentajes, fechas UTC, duraciones, valores, descarga CSV, etiquetas de columna y nodos.

Función exportada	Parámetros (resumen)
formatearNumero	value

formatearPorcentaje	value
formatearFecha	value
formatearFechaCorta	value
formatearDuracion	ms
formatearValor	value
descargarCsv	nombre, columnas, filas
colorPorTipo	tipo
etiquetaNodo	nodo
tituloColumna	columna

src/workers/csvValidation.worker.js

Recibe File[] por postMessage, valida extensión/tamaño, decodifica UTF-8 estricto, ejecuta prepareCsv y devuelve resultados o error controlado.
Dependencias importadas: ../utils/csvValidation.

13. Pruebas y verificación

```
# Backend, desde proyecto-final-db2/
./mvnw test
./mvnw test package
./mvnw dependency:tree
# Frontend, desde frontend-proyecto-final-db2/
npm ci
npm test
npm run build
npm audit
```

Pruebas Java presentes localmente: 40 casos en la última tanda, 39 aprobados y una integración MongoQueryIntegrationTest omitida que requiere TEST_MONGO_URI y usa una base temporal aislada eliminada al finalizar. Frontend: 10 pruebas de normalización CSV con node:test. Los resultados son evidencia histórica, no sustituyen ejecutar la suite en cada entorno. Este manual no implica haber repetido conexiones a las bases ni cargas de datos.
No configurar TEST_MONGO_URI con credenciales en comandos publicados; proporcionar el valor por entorno privado. Aunque la integración usa una base con prefijo etl_query_test_, el usuario de pruebas necesita permisos para crear y eliminar esa base. No ejecutar fixtures contra la base operativa.

- Verificar GET /api/salud sin escrituras y disponibilidad independiente de cada base.
- Probar selección local de CSV alterado, nulos, estados desconocidos y comillas corruptas: no debe enviarse el lote.
- Solo con autorización cargar un dataset controlado, consultar el job y verificar invariantes de métricas.

- Reenviar el mismo dataset debe incrementar duplicados, no evaluaciones nuevas idénticas; revisar el grafo.
- Simular fallos de drivers con mocks en pruebas, no deteniendo bases compartidas.
- Un aviso de tamaño del bundle Vite no es una falla de compilación; dividir imports si afecta rendimiento. Revisar vulnerabilidades de npm sin aplicar actualizaciones incompatibles automáticamente.

14. Despliegue y operación

Desarrollo: Vite sirve la SPA y proxifica /api. Producción: compilar dist, servir archivos mediante servidor estático con fallback de rutas a index.html y configurar un reverse proxy de /api al backend. El proxy de vite.config.js solo se define para desarrollo y no configura un reverse proxy productivo. El backend ejecuta JAR con JDK compatible; Compose actual no lo conteneriza.

- Terminar HTTPS en el proxy; restringir CORS y aplicar límites de cuerpo/tiempo coherentes con multipart y el timeout Axios.
- No publicar 27017, 7474 ni 7687 a Internet. Usar red privada, firewall, usuarios mínimos y secretos administrados.
- Gestionar backend con un supervisor (systemd/orquestador), shutdown ordenado y logs centralizados; ajustar memoria del JVM y Neo4j según volumen medido.
- Respaldar MongoDB y Neo4j con herramientas oficiales y política de recuperación probada; preservar también consultas y auditoría.
- No usar docker compose down -v como operación habitual. Un volumen persistente no sustituye un backup.
- Separar credenciales de ejecución, administración y pruebas; rotar secretos fuera del código.
- La pérdida del proceso elimina jobs en memoria y la cola del executor. Los reportes persistidos pueden quedar EN_PROCESO tras un reinicio; reconciliar y reintentar con criterio operacional.
- Vigilar disco temporal, latencia por lote, rechazados, pendientes, sincronización de grafo, tamaño de documentos embebidos y tiempos de consulta.

Síntoma	Diagnóstico y acción segura
Puerto backend ocupado	Identificar proceso con ss -ltnp o lsof -i :8080. No matar el IDE ni procesos ajenos; detener la instancia propia o cambiar SERVER_PORT y el proxy.
Vite no inicia	Completar API_PROXY_TARGET y FRONTEND_PORT; validar puerto entero 1..65535; reiniciar tras editar .env.
MongoDB no disponible	Revisar docker compose ps y logs; comprobar URI, base explícita, autenticación y red. No imprimir URI privada en capturas.
Neo4j no conecta	Esperar disponibilidad; verificar Bolt y credenciales contra el volumen persistido.

	HTTP 7474 disponible no prueba por sí solo autorización Bolt.
Carga ERROR con procesados>0	Hay posible persistencia parcial: revisar detalles, pendientes y sincronizadosNeo4j; no asumir rollback. Recuperar conectividad y reenviar archivo aprobado.
413 o rechazo por tamaño	Distinguir límite frontend 20 MB y multipart backend/proxy. Dividir CSV manteniendo encabezados y registros entrecomillados íntegros; no cortar por bytes arbitrarios.
Rutas React fallan al refrescar	Configurar fallback del servidor estático a index.html, sin interceptar rutas /api.
C6/fichas no reflejan escritura administrativa	Las consultas personalizadas no sincronizan vistas ni bases. Planificar reconstrucción y reconciliación explícitas.
Pruebas no existen al clonar	Están parcialmente ignoradas por Git: revisar distribución de archivos antes de ejecutar comandos o reportar cobertura.

15. Limitaciones y evolución

La implementación es adecuada como proyecto académico local, no una garantía de endurecimiento productivo. Los siguientes elementos son recomendaciones, no funcionalidades presentes:

- Cola durable y patrón outbox/reconciliación para sincronización eventual MongoDB-Neo4j; actualmente dos escrituras separadas.
- Control de concurrencia para reconstrucción de aspirantes: hay dos hilos ETL y $out puede competir entre cargas; serializar reconstrucción o rediseñar la proyección.
- Política de corrección de evaluaciones para conciliar hash de fila documental con clave lógica de intento en grafo.
- Autenticación/roles, auditoría de administradores y rate limits para todas las rutas de escritura; CORS y clave de consultas no sustituyen estas capas.
- Paginación/streaming de resultados voluminosos, límites de complejidad y cuotas para consultas administrativas.
- Retención o expulsión del mapa de jobs y políticas de auditoría; actualmente los jobs permanecen en memoria.
- Revisar tamaño máximo BSON de 16 MiB para aspirantes con historial embebido muy grande.
- Validación definitiva de columnas, codificaciones, volumen e interpretación con el dataset oficial; los CSV sintéticos no sustentan conclusiones poblacionales.

16. Referencias y trazabilidad

Referencia	Uso
pom.xml	Versiones Java y dependencias backend.
package.json / package-lock.json	Versiones y scripts frontend.
docker-compose.yml	Servicios, puertos, volúmenes y healthchecks.
vite.config.js / api/client.js	Proxy, puerto, URL API y timeout.
src/main/java/	Inventario de responsabilidades y firmas.
documentacion/diagrama_estructura.wsd	Arquitectura y comunicación explícita.
documentacion/diagrama_de_clases.wsd	Relaciones UML y clasificadores.
docs/	Markdown local de soporte; ignorado por Git.

17. Consultas definidas

Las consultas definidas tanto MongoDB y Cypher están en el informe adjunto a este documento.

18. Repositorio

El repositorio del proyecto se encuentra en: [proyecto-final-db2].